SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%
Dr.V.Parthipan

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I Vaishnavee Bhavani Vijayakumar (1912571071) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

1.1 Architecture of a Servlet-Based Application

A servlet-based application follows a multi-tier client–server architecture built on the HTTP request–response model. It has three logical tiers:

- **Client tier.** The student's web browser displays the HTML registration form and sends the entered data to the server as an HTTP request.
- **Web tier.** A web server with a servlet container (for example Apache Tomcat) receives the request. The container converts it into an `HttpServletRequest` object, creates an `HttpServletResponse` object, selects the correct servlet from the URL pattern, and runs it on a thread taken from its thread pool. The container also manages the servlet life cycle, sessions, filters and security.
- **Data tier.** The servlet may call business classes and JDBC code to store the registration in a database and then returns a dynamically generated HTML page.

The processing of one registration request takes place in the following sequence:

1. The student fills in the form and the browser sends an HTTP request to the URL `/register`.
2. The container finds the servlet mapped to that URL (through the `@WebServlet` annotation or *web.xml*) and loads and initialises it if it is not already loaded.
3. The container creates the request and response objects and calls the servlet's `service()` method on a new thread.
4. `service()` checks the HTTP method and calls `doGet()` or `doPost()`.
5. The servlet reads the form fields with `getParameter()`, validates them, applies the business logic and writes an HTML page through `response.getWriter()`.
6. The container sends the response to the browser and the thread returns to the pool. The servlet object itself stays in memory for the next request.

An important feature of this architecture is that the container creates only **one instance** of each servlet and serves all users concurrently with multiple threads. Instance variables are therefore shared between requests and must be thread-safe, which is why the implementation below uses `AtomicInteger` for its counters.

1.2 Design and Implementation

The registration form collects the name, register number, e-mail, department, year and password, and submits them to the servlet mapped to `/register`. The servlet overrides both `doGet()` and `doPost()` and delegates to one common method, `process()`, so that the same validation and response logic is used for both methods. It validates the required fields and the e-mail format (returning HTTP status 400 for invalid data), escapes every value before writing it back to the page to prevent script injection, and reports how the data travelled so that the two methods can be compared. The program was compiled with JDK 21 against the Jakarta Servlet 6.0 API and deployed on Apache Tomcat 10.1 under the context path `/college`.

register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Registration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h1>Student Registration</h1>
    <form action="register" method="post">
      <label>Full Name</label>
      <input type="text" name="name" required>

      <label>Register Number</label>
      <input type="text" name="regNo" required>

      <label>Email</label>
      <input type="email" name="email" required>

      <label>Department</label>
      <select name="department">
        <option>Computer Science</option>
        <option>Information Technology</option>
        <option>Electronics</option>
        <option>Mechanical</option>
      </select>

      <label>Year</label>
      <select name="year">
        <option>I</option><option>II</option><option>III</option><option>IV</option>
      </select>

      <label>Password</label>
      <input type="password" name="password" required>

      <button type="submit">Register</button>
    </form>
  </div>
</body>
</html>
```

StudentRegistrationServlet.java

```
package com.simats.registration;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
```

```
import java.io.PrintWriter;
import java.util.concurrent.atomic.AtomicInteger;

@WebServlet(urlPatterns = "/register", loadOnStartup = 1)
public class StudentRegistrationServlet extends HttpServlet {

    private final AtomicInteger requestCount = new AtomicInteger();
    private final AtomicInteger applicationNo = new AtomicInteger(1000);

    @Override
    public void init() throws ServletException {
        log("[init] StudentRegistrationServlet initialised");
    }

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        log("[service] " + req.getMethod() + " request no. " + requestCount.incrementAndGet());
        super.service(req, resp);              // dispatches to doGet() or doPost()
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        if (req.getQueryString() == null) {    // plain visit: show the form
            resp.sendRedirect("register.html");
            return;
        }
        process(req, resp, "GET");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws IOException {
        process(req, resp, "POST");
    }

    private void process(HttpServletRequest req, HttpServletResponse resp, String method)
            throws IOException {
        req.setCharacterEncoding("UTF-8");
        resp.setContentType("text/html;charset=UTF-8");
        PrintWriter out = resp.getWriter();

        String name  = value(req, "name");
        String regNo = value(req, "regNo");
        String email = value(req, "email");
        String dept  = value(req, "department");
        String year  = value(req, "year");

        if (name.isEmpty() || regNo.isEmpty() || !email.matches("[^@\\s]+@[^@\\s]+\\.[^@\\s]+")) {
            resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
            out.println(page("<h1 class='err'>Registration failed</h1>"
                    + "<p>Name, register number and a valid e-mail are required.</p>"));
            return;
        }

        String travelled = method.equals("GET")
                ? req.getRequestURI() + "?" + req.getQueryString()
                : req.getRequestURI() + "  (data sent in the request body, "
                        + req.getContentLength() + " bytes)";

        out.println(page("""
                <h1 class='ok'>Registration successful</h1>
                <table>
                  <tr><td>Application No.</td><td>APP%d</td></tr>
                  <tr><td>Name</td><td>%s</td></tr>
                  <tr><td>Register No.</td><td>%s</td></tr>
                  <tr><td>Email</td><td>%s</td></tr>
                  <tr><td>Department</td><td>%s, Year %s</td></tr>
                  <tr><td>HTTP method</td><td>%s</td></tr>
                  <tr><td>Data travelled in</td><td>%s</td></tr>
                </table>
```

```
                """.formatted(applicationNo.incrementAndGet(), esc(name), esc(regNo),
                        esc(email), esc(dept), esc(year), method, esc(travelled))));
    }

    @Override
    public void destroy() {
        log("[destroy] StudentRegistrationServlet destroyed after "
                + requestCount.get() + " requests");
    }

    private static String value(HttpServletRequest req, String field) {
        String v = req.getParameter(field);
        return v == null ? "" : v.trim();
    }

    private static String esc(String s) {           // prevents HTML/script injection
        return s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("\"", "&quot;");
    }

    private static String page(String body) {
        return "<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Registration</title>"
                + "<link rel='stylesheet' href='style.css'></head>"
                + "<body><div class='card'>" + body + "</div></body></html>";
    }
}
```

Output

Output 1: the registration form filled in by the student and submitted with the POST method.

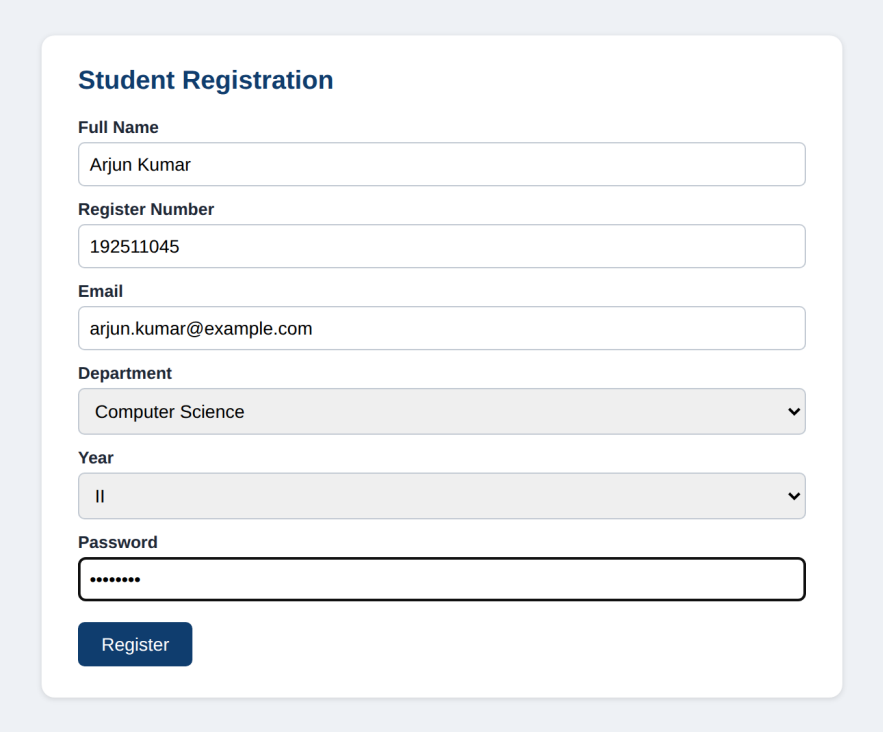


Figure 1: Registration form (register.html) with details entered

Output 2: response of `doPost()`. The data travelled in the request body, so the URL contains no form values.

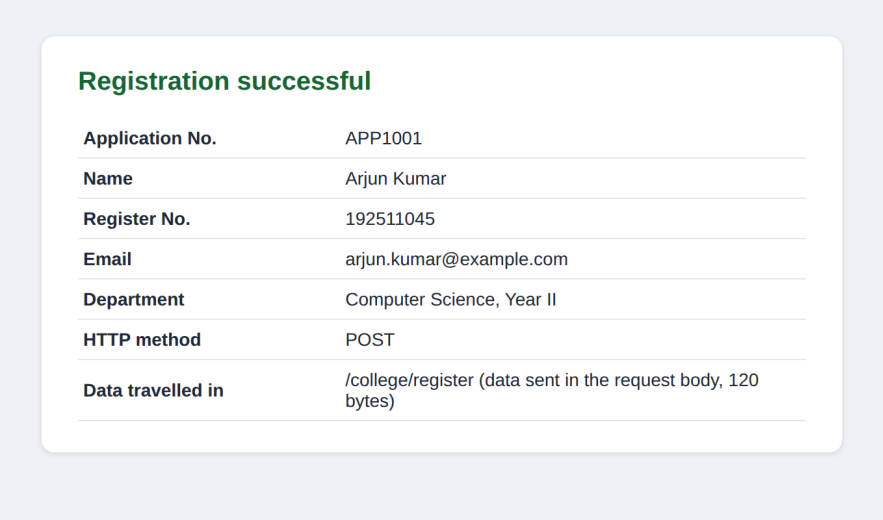


Figure 2: Result of submitting the form with the POST method

Output 3: the same data sent with the GET method by typing the query string in the address bar. The servlet processes it in `doGet()`, but every field, including the password, is now part of the URL.

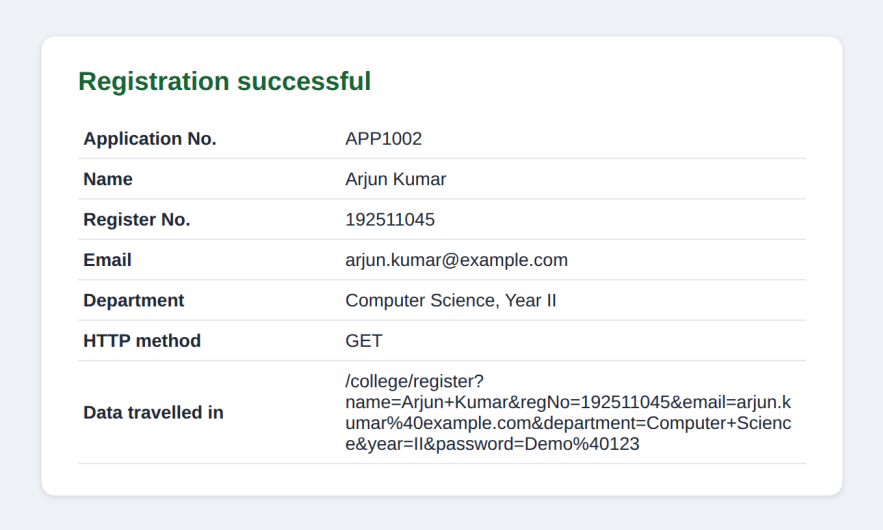


Figure 3: Result of sending the data with the GET method

A request with an empty name and an invalid e-mail address was also sent to the servlet; it was rejected with HTTP status 400 (Bad Request), as designed.

1.3 Servlet Life Cycle

The life cycle of a servlet is controlled completely by the servlet container and consists of loading and instantiation followed by three life-cycle methods defined in the `jakarta.servlet.Servlet` interface.

- **Loading and instantiation.** The container loads the servlet class and creates a single instance. This happens when the first request arrives, or when the application starts if `loadOnStartup` is set, as in the program above.

- **init().** Called exactly once, after the instance is created and before any request is served. It is used for one-time work such as reading initialisation parameters, opening a connection pool or loading configuration. If it throws an exception, the servlet is not put into service.
- **service().** Called by the container for every request, each on its own thread. It reads the HTTP method and dispatches to `doGet()`, `doPost()` or another handler. It is normally not overridden; here it is overridden only to log the request and then calls `super.service()`.
- **destroy().** Called exactly once when the container removes the servlet, for example at application undeployment or server shutdown, after the running service threads have finished. It is used to release resources such as database connections, files and threads.

The server log below was produced by the servlet in Section 1.2. `init()` ran once when the application was deployed, `service()` ran for each of the three requests (one POST and two GET), and `destroy()` ran once when Tomcat was shut down.

Output

```
$ grep -E "\[(init|service|destroy)\]" server.log
08:02:07 INFO  com.simats.registration.StudentRegistrationServlet: [init]
StudentRegistrationServlet initialised
08:02:08 INFO  com.simats.registration.StudentRegistrationServlet: [service] POST request no. 1
08:02:08 INFO  com.simats.registration.StudentRegistrationServlet: [service] GET request no. 2
08:02:09 INFO  com.simats.registration.StudentRegistrationServlet: [service] GET request no. 3
08:02:09 INFO  com.simats.registration.StudentRegistrationServlet: [destroy]
StudentRegistrationServlet destroyed after 3 requests
```

Figure 4: Servlet life-cycle messages written to the server log

1.4 Choice of Method for Sensitive Data

The **POST** method is the appropriate method for sensitive data such as passwords, and it is therefore the method used by the registration form. The reasons are as follows.

- **Visibility.** GET appends the form data to the URL as a query string, so it is displayed in the address bar and can be seen by anyone looking at the screen. Figure 3 shows the password in the URL. POST sends the data in the body of the request, so nothing appears in the URL.
- **History, logs and sharing.** URLs are stored in browser history, bookmarks, web server access logs, proxy logs and the Referer header sent to other sites, and they can be copied and shared. Data in a POST body is not recorded in any of these places.
- **Size.** URLs have practical length limits (Tomcat’s default header limit is 8 KB), whereas POST can carry much larger payloads.
- **Purpose of the request.** GET is meant for safe, repeatable, cacheable retrieval of data and can be bookmarked or pre-fetched by browsers and crawlers. A registration changes the state of the server, so it should use POST, which is not cached or silently repeated.

POST by itself does not encrypt the data. Sensitive data must also be sent over HTTPS so that the request body is protected while it travels across the network. In the servlet above `doGet()` is kept only to show the difference between the two methods; a production system would accept registration data only through `doPost()` and would reserve GET for non-sensitive requests such as searching or viewing a page.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

2.1 How Session Tracking Works

HTTP is a stateless protocol: every request is independent, and the server does not remember that the request for the second page came from the user who logged in on the first page. Session tracking is the technique of giving each user an identity that is carried with every request so that the server can link the requests together. The common techniques are cookies, URL rewriting, hidden form fields and the `HttpSession` API. In the Servlet API, `HttpSession` is the standard mechanism, and by default it uses a cookie to carry the session identifier. It works as follows:

1. When the user logs in successfully, the servlet calls `request.getSession(true)`. The container creates an `HttpSession` object in server memory and gives it a long, random, unique session ID.
2. The container sends this ID to the browser in a `Set-Cookie: JSESSIONID=...` response header.
3. The browser stores the cookie and automatically returns it in the `Cookie` header of every later request to the same application.
4. On each request the container reads the ID, finds the matching session object and returns it from `request.getSession(false)`. The attributes stored in the session (user name, login time and so on) are therefore available on every page.
5. The session ends when the application calls `invalidate()` (logout) or when the user is idle longer than the timeout set with `setMaxInactiveInterval()`. If cookies are disabled, the container can fall back to URL rewriting (`;jsessionid=...`) when the application encodes its URLs with `response.encodeURL()`.

2.2 Design and Implementation

The system consists of five servlet-side components and one helper class:

- `LoginServlet` shows the login form, verifies the credentials, creates the session and sets the cookie.
- `AuthFilter` is a servlet filter that protects the private pages. A request without a valid session is redirected to the login page.
- `DashboardServlet` and `ProfileServlet` are two private pages that read the shared session data.
- `LogoutServlet` invalidates the session.
- `Html` contains the page layout and the HTML-escaping routine used by all servlets.

The main design decisions are these. Authentication state (user name, login time, page counter) is kept in the `HttpSession` so that it stays on the server and the browser only holds a random ID. A separate persistent cookie, `lastUser`, stores only the non-sensitive user name for seven days to

pre-fill the login form. Both cookies are marked `HttpOnly` so that JavaScript cannot read them. The old session is invalidated before a new one is created at login, which prevents session fixation. `getSession(false)` is used wherever a session must not be created by accident. The session expires after 15 minutes of inactivity, and logout uses POST so that it cannot be triggered by a simple link or image. The demonstration user names and passwords are hard-coded for simplicity; a real system would verify hashed passwords stored in a database and would run on HTTPS with the `Secure` cookie attribute. The application was deployed on Tomcat 10.1 under the context path `/portal`.

LoginServlet.java

```
package com.simats.session;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;
import java.util.Map;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    // Demo users only. A real system reads hashed passwords from a database.
    private static final Map<String, String> USERS =
            Map.of("student01", "Simats@123", "faculty01", "Parthi@123");

    private static final int SESSION_TIMEOUT = 15 * 60;        // 15 minutes of inactivity
    private static final int COOKIE_MAX_AGE  = 7 * 24 * 60 * 60; // 7 days

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);            // never create a session here
        if (session != null && session.getAttribute("user") != null) {
            resp.sendRedirect(req.getContextPath() + "/dashboard");
            return;
        }

        String lastUser = "";                                   // read the cookie sent by browser
        if (req.getCookies() != null) {
            for (Cookie c : req.getCookies()) {
                if (c.getName().equals("lastUser")) lastUser = c.getValue();
            }
        }

        String message = switch (String.valueOf(req.getParameter("msg"))) {
            case "error"     -> "<p class='err'>Invalid username or password.</p>";
            case "required"  -> "<p class='err'>Please log in to continue.</p>";
            case "loggedout" -> "<p class='ok'>You have been logged out.</p>";
            default          -> "";
        };

        resp.setContentType("text/html;charset=UTF-8");
        resp.getWriter().println(Html.page(req, """
                <h1>Student Portal Login</h1>%s
                <form action="login" method="post">
                  <label>Username</label>
                  <input type="text" name="username" value="%s" required>
                  <label>Password</label>
                  <input type="password" name="password" required>
                  <button type="submit">Log in</button>
                </form>
                """.formatted(message, Html.esc(lastUser))));
    }
```

```
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        if (user == null || pass == null || !pass.equals(USERS.get(user))) {
            resp.sendRedirect(req.getContextPath() + "/login?msg=error");
            return;
        }

        HttpSession old = req.getSession(false);
        if (old != null) old.invalidate();                    // prevents session fixation

        HttpSession session = req.getSession(true);           // new JSESSIONID is issued
        session.setAttribute("user", user);
        session.setAttribute("loginTime", Html.time(System.currentTimeMillis()));
        session.setAttribute("pageViews", 0);
        session.setMaxInactiveInterval(SESSION_TIMEOUT);

        Cookie remember = new Cookie("lastUser", user);       // persistent cookie
        remember.setMaxAge(COOKIE_MAX_AGE);
        remember.setHttpOnly(true);                           // not readable by JavaScript
        remember.setPath(req.getContextPath().isEmpty() ? "/" : req.getContextPath());
        resp.addCookie(remember);

        resp.sendRedirect(req.getContextPath() + "/dashboard");
    }
}
```

AuthFilter.java

```
package com.simats.session;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

/** Protects every private page: no valid session means redirect to the login page. */
@WebFilter({"/dashboard", "/profile"})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);          // do not create a session here
        if (session == null || session.getAttribute("user") == null) {
            resp.sendRedirect(req.getContextPath() + "/login?msg=required");
            return;
        }
        resp.setHeader("Cache-Control", "no-store");          // Back button shows no old pages
        chain.doFilter(request, response);
    }
}
```

DashboardServlet.java

```
package com.simats.session;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);            // AuthFilter guarantees it exists
        int views = (Integer) session.getAttribute("pageViews") + 1;
        session.setAttribute("pageViews", views);

        resp.setContentType("text/html;charset=UTF-8");
        resp.getWriter().println(Html.page(req, """
                <h1>Dashboard</h1>
                <p>Welcome, <b>%s</b>.</p>
                <table>
                  <tr><td>Session ID</td><td>%s</td></tr>
                  <tr><td>Session created</td><td>%s</td></tr>
                  <tr><td>Last accessed</td><td>%s</td></tr>
                  <tr><td>Idle timeout</td><td>%d minutes</td></tr>
                  <tr><td>Pages viewed</td><td>%d</td></tr>
                </table>
                <p><a href="profile">Go to Profile</a></p>
                <form action="logout" method="post"><button type="submit">Log out</button></form>
                """.formatted(Html.esc((String) session.getAttribute("user")), session.getId(),
                        Html.time(session.getCreationTime()),
                        Html.time(session.getLastAccessedTime()),
                        session.getMaxInactiveInterval() / 60, views)));
    }
}
```

ProfileServlet.java

```
package com.simats.session;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

@WebServlet("/profile")
public class ProfileServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);
        int views = (Integer) session.getAttribute("pageViews") + 1;
        session.setAttribute("pageViews", views);

        resp.setContentType("text/html;charset=UTF-8");
        resp.getWriter().println(Html.page(req, """
                <h1>My Profile</h1>
                <table>
                  <tr><td>Username</td><td>%s</td></tr>
                  <tr><td>Logged in at</td><td>%s</td></tr>
                  <tr><td>Session ID</td><td>%s</td></tr>
                  <tr><td>Pages viewed</td><td>%d</td></tr>
                </table>
                <p>The same session is used on every page, so no second login is needed.</p>
```

```
                <p><a href="dashboard">Back to Dashboard</a></p>
                """.formatted(Html.esc((String) session.getAttribute("user")),
                        session.getAttribute("loginTime"), session.getId(), views()));
    }
}
```

LogoutServlet.java

```
package com.simats.session;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate();                               // server-side data is destroyed
        }
        resp.sendRedirect(req.getContextPath() + "/login?msg=loggedout");
    }
}
```

Html.java

```
package com.simats.session;

import jakarta.servlet.http.HttpServletRequest;

import java.time.Instant;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;

/** Small helper shared by all servlets: HTML escaping and the common page layout. */
final class Html {

    private static final DateTimeFormatter TIME =
            DateTimeFormatter.ofPattern("dd MMM yyyy, HH:mm:ss").withZone(ZoneId.systemDefault());

    private Html() { }

    static String esc(String s) {
        return s.replace("&", "&amp;").replace("<", "&lt;")
                .replace(">", "&gt;").replace("\"", "&quot;");
    }

    static String time(long epochMillis) {
        return TIME.format(Instant.ofEpochMilli(epochMillis));
    }

    static String page(HttpServletRequest req, String body) {
        return "<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Student Portal</title>"
                + "<link rel='stylesheet' href='" + req.getContextPath() + "/style.css'></head>"
                + "<body><div class='card'>" + body + "</div></body></html>";
    }
}
```

Output

Output 1: the login page when the user visits for the first time. No cookie exists yet, so the user name field is empty.

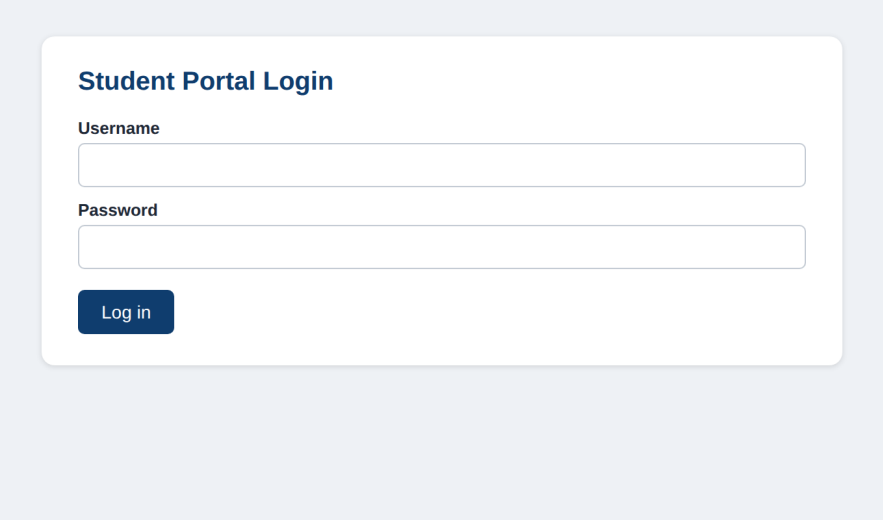


Figure 5: Login page (first visit)

Output 2: the dashboard after a successful login. The page shows the session ID created by the container and the first page view.

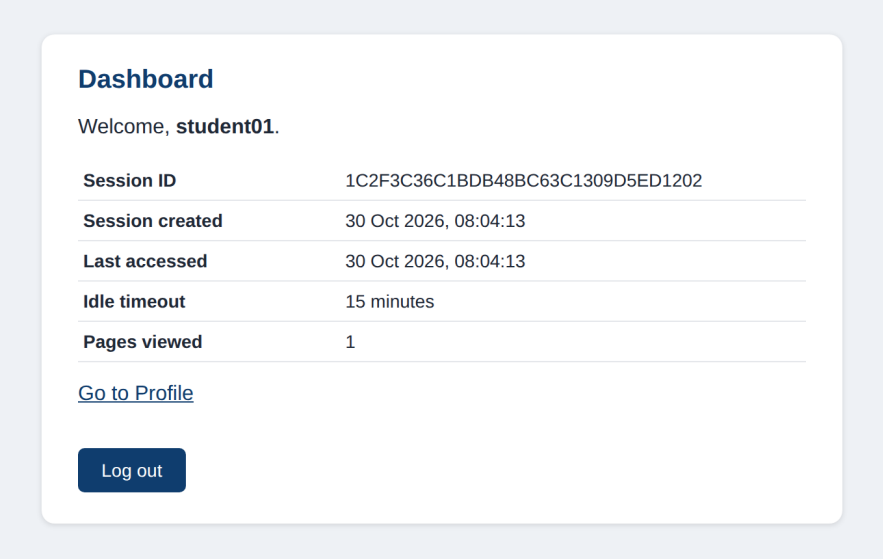


Figure 6: Dashboard displayed after login

Output 3: the profile page, reached by following a link. The session ID is the same and the page counter has increased to 2, which shows that the session was maintained across pages without a second login.

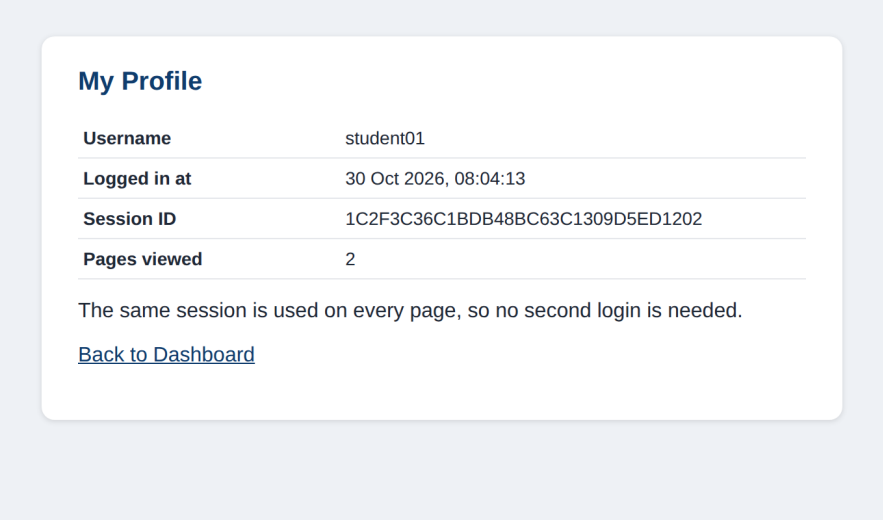


Figure 7: Profile page using the same session

Output 4: the page after pressing the logout button. The session has been destroyed on the server, but the persistent `lastUser` cookie is still sent by the browser, so the user name is pre-filled.

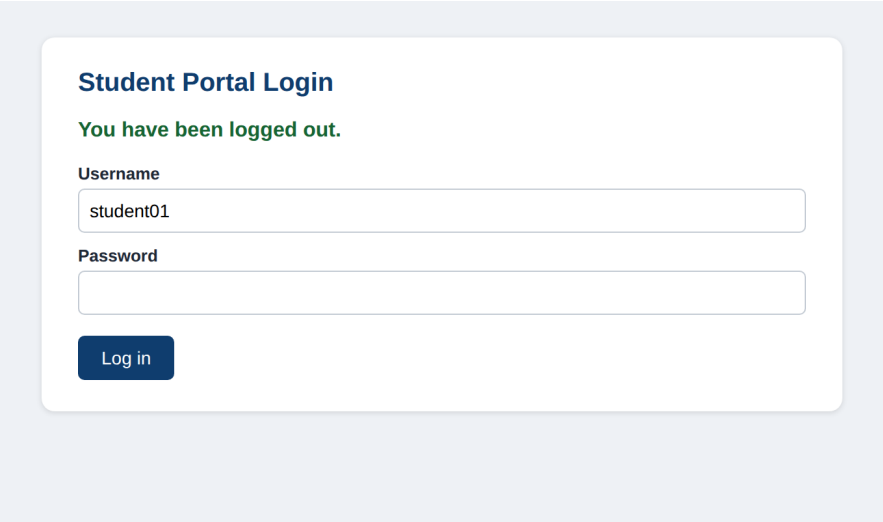


Figure 8: Login page after logging out

Output 5: an attempt to open the dashboard after logout. Because the session no longer exists, `AuthFilter` redirects the request to the login page.

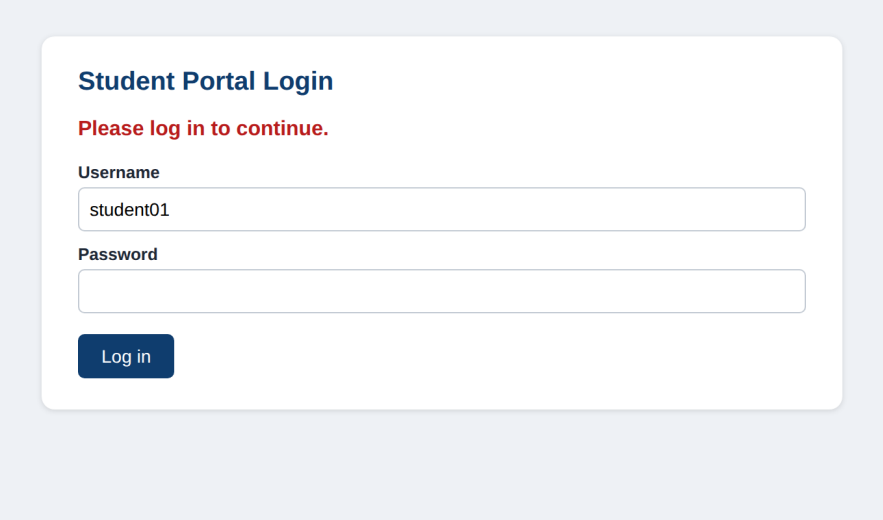


Figure 9: Private page requested without a valid session

Output 6: the HTTP messages exchanged during the same sequence. The first request has no cookie and is redirected. At login the server sends two `Set-Cookie` headers, one for the session ID and one for `lastUser`. The browser returns both in the `Cookie` header of the next requests, and the logout request ends the session.

```
$ curl -v http://localhost:8086/portal/dashboard
< HTTP/1.1 302
< Location: /portal/login?msg=required

$ curl -v -c cookies.txt -b cookies.txt -d "username=student01&password=Simats@123" .../login
< HTTP/1.1 302
< Set-Cookie: JSESSIONID=57511E7E727D22C8F73A9D797B2508E1; Path=/portal; HttpOnly
< Set-Cookie: lastUser=student01; Max-Age=604800; Expires=Fri, 06 Nov 2026 08:04:12 GMT;
Path=/portal; HttpOnly
< Location: /portal/dashboard

$ curl -v -c cookies.txt -b cookies.txt http://localhost:8086/portal/dashboard
> Cookie: JSESSIONID=57511E7E727D22C8F73A9D797B2508E1; lastUser=student01
< HTTP/1.1 200

$ curl -v -c cookies.txt -b cookies.txt -X POST http://localhost:8086/portal/logout
> Cookie: JSESSIONID=57511E7E727D22C8F73A9D797B2508E1; lastUser=student01
< HTTP/1.1 302
< Location: /portal/login?msg=loggedout
```

Figure 10: Request and response headers showing session cookies

2.3 Session-Based Tracking Compared with Cookies

Cookies and sessions are closely related. A cookie is a small piece of data stored in the browser. An `HttpSession` is an object stored on the server, and it is linked to the browser by a cookie that contains only the session ID. The two approaches differ in the following ways.

- **Place of storage.** Session data stays on the server. Cookie data is stored on the client machine.
- **Security.** The client can read and edit cookie data, so passwords or user roles must never be stored in a cookie. With a session the browser sees only a random ID, so sensitive data stays on the server. The session ID itself must still be protected with HTTPS, the `Secure` and `HttpOnly` attributes and regeneration at login.

- **Type and size of data.** A cookie holds only a short text value, about 4 KB per cookie, with a limited number of cookies per domain. A session can hold any Java object and is limited only by server memory.
- **Lifetime.** A cookie can persist for days or years when `setMaxAge()` is used, and it survives restarting the browser. A session lasts until logout, until the idle timeout, or until the server restarts, and its ID cookie normally disappears when the browser closes.
- **Server load and scalability.** Sessions use server memory and need session replication, sticky routing or an external store when the application runs on several servers. Cookies cost the server nothing because the client keeps the data.
- **Typical use.** Sessions suit login state, shopping carts and other private, changing data. Cookies suit small preferences such as language, theme and a remembered user name.

The implementation above uses both: the session holds the authenticated state, while the cookie remembers the user name for convenience. Session tracking is therefore normally the better choice for authentication, and plain cookies are better for long-lived, non-sensitive preferences.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application. Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

3.1 JDBC Architecture

Java Database Connectivity (JDBC) is the standard Java API, in the `java.sql` and `javax.sql` packages, for accessing relational databases. It separates the application from the database product by using layers:

- **Java application.** A servlet or any Java program that uses only the JDBC interfaces such as `Connection`, `Statement`, `PreparedStatement` and `ResultSet`.
- **JDBC API and DriverManager.** The API defines the interfaces. `DriverManager` (or a `DataSource` in a connection pool) selects a registered driver that accepts the connection URL.
- **JDBC driver.** A vendor-supplied library that implements the interfaces and converts JDBC calls into the protocol of one particular database.
- **Database.** The DBMS (MySQL, Oracle, PostgreSQL, H2 and so on) that executes the SQL and returns the results.

JDBC drivers are classified into four types. **Type 1** (JDBC–ODBC bridge) translates calls to ODBC and was removed in Java 8. **Type 2** (native API) uses the database client library installed on the machine. **Type 3** (network protocol) talks to a middleware server that connects to the database. **Type 4** (thin, pure Java) talks directly to the database in its own network protocol. Type 4 drivers, such as MySQL Connector/J and the H2 driver used here, are the most widely used because they are portable and need no client installation.

3.2 Steps in Database Connectivity

1. **Load the driver.** `Class.forName("driverClass")` loads and registers the driver. Since JDBC 4.0 drivers on the class path register themselves automatically, but this step is kept for clarity and for older drivers.
2. **Establish the connection.** `DriverManager.getConnection(url, user, password)` returns a `Connection`. The URL identifies the database, for example `jdbc:mysql://localhost:3306/ecommerce`.
3. **Create a statement.** `createStatement()` for fixed SQL, or `prepareStatement(sql)` for parameterised SQL.
4. **Execute the SQL.** `executeUpdate()` or `executeBatch()` for INSERT, UPDATE, DELETE and DDL, and `executeQuery()` for SELECT.
5. **Process the results.** Move through the `ResultSet` with `next()` and read the columns with `getInt()`, `getString()`, `getBigDecimal()` and similar methods.
6. **Handle exceptions and manage the transaction.** Catch `SQLException`, commit on success and roll back on failure.

7. **Close the resources.** Close the `ResultSet`, the statement and the connection. The program below uses try-with-resources so this happens automatically.

3.3 Design and Implementation

The application stores product details in a table named `products` with the columns `id` (primary key), `name`, `category`, `price` (`DECIMAL(10,2)`, which avoids rounding errors in money values) and `stock`. The program creates the table, inserts five products in one transaction using a batched `PreparedStatement`, retrieves all products with a SELECT query, runs a second parameterised query (Electronics products under Rs. 2000) and finally tries to insert a duplicate product ID to show exception handling. Parameters are bound to `?` placeholders rather than joined into the SQL text, which prevents SQL injection. The program was run with the H2 database (version 2.2.220) and its pure-Java Type 4 driver; to use MySQL, only the driver class, the URL and the credentials in the three constants at the top need to be changed, and the MySQL Connector/J library added to the class path. The same code can be called from a servlet in a web application.

ProductJdbcDemo.java

```
package com.simats.jdbc;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ProductJdbcDemo {

    // Change these three values to use MySQL instead:
    //   jdbc:mysql://localhost:3306/ecommerce  and  com.mysql.cj.jdbc.Driver
    private static final String DRIVER = "org.h2.Driver";
    private static final String URL    = "jdbc:h2:./data/ecommerce";
    private static final String USER   = "sa";
    private static final String PASS   = "";

    public static void main(String[] args) {
        try {
            Class.forName(DRIVER);                                    // Step 1: load the driver
        } catch (ClassNotFoundException e) {
            System.err.println("JDBC driver not found: " + e.getMessage());
            return;
        }

        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {   // Step 2: connect
            System.out.println("Connected to " + con.getMetaData().getDatabaseProductName()
                    + " " + con.getMetaData().getDatabaseProductVersion());

            createTable(con);
            insertProducts(con);
            showAllProducts(con);
            showProductsByCategory(con, "Electronics", new BigDecimal("2000"));
            insertDuplicate(con);
        } catch (SQLException e) {
            printSqlError("Database error", e);
        }                                                             // connection closed here
    }

    private static void createTable(Connection con) throws SQLException {
        String ddl = "CREATE TABLE IF NOT EXISTS products ("
                + "id INT PRIMARY KEY, name VARCHAR(60) NOT NULL, category VARCHAR(30), "
                + "price DECIMAL(10,2) NOT NULL, stock INT DEFAULT 0)";
        try (Statement st = con.createStatement()) {                  // Step 3: create statement
```

```
        st.execute(ddl);                                        // Step 4: execute
        System.out.println("Table 'products' is ready.");
    }
}

private static void insertProducts(Connection con) throws SQLException {
    String sql = "INSERT INTO products (id, name, category, price, stock) "
            + "VALUES (?, ?, ?, ?, ?)";
    Object[][] rows = {
        {101, "Wireless Mouse",       "Electronics",    "799.00",  50},
        {102, "Bluetooth Headphones", "Electronics",    "2499.00", 30},
        {103, "Cotton T-Shirt",       "Clothing",       "499.00",  120},
        {104, "Steel Water Bottle",   "Home & Kitchen", "349.00",  75},
        {105, "Laptop Backpack",      "Accessories",    "1299.00", 40}
    };

    con.setAutoCommit(false);                                   // start a transaction
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        for (Object[] r : rows) {
            ps.setInt(1, (Integer) r[0]);
            ps.setString(2, (String) r[1]);
            ps.setString(3, (String) r[2]);
            ps.setBigDecimal(4, new BigDecimal((String) r[3]));
            ps.setInt(5, (Integer) r[4]);
            ps.addBatch();
        }
        int[] counts = ps.executeBatch();
        con.commit();                                           // all rows saved together
        System.out.println(counts.length + " products inserted and committed.");
    } catch (SQLException e) {
        con.rollback();                                         // undo the partial insert
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}

private static void showAllProducts(Connection con) throws SQLException {
    String sql = "SELECT id, name, category, price, stock FROM products ORDER BY id";
    System.out.println("\nAll products:");
    System.out.printf("%-5s %-22s %-16s %10s %6s%n",
            "ID", "Name", "Category", "Price", "Stock");
    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {                 // Step 5: run the query
        while (rs.next()) {                                     // Step 6: process result
            System.out.printf("%-5d %-22s %-16s %10.2f %6d%n",
                    rs.getInt("id"), rs.getString("name"), rs.getString("category"),
                    rs.getBigDecimal("price"), rs.getInt("stock"));
        }
    }
}

private static void showProductsByCategory(Connection con, String category,
        BigDecimal maxPrice) throws SQLException {
    String sql = "SELECT name, price FROM products "
            + "WHERE category = ? AND price < ? ORDER BY price";
    System.out.println("\n" + category + " products costing less than Rs. " + maxPrice + ":");
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, category);          // values are bound to the ? placeholders,
        ps.setBigDecimal(2, maxPrice);      // never concatenated: prevents SQL injection
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                System.out.printf("  %-22s Rs. %.2f%n",
                        rs.getString("name"), rs.getBigDecimal("price"));
            }
        }
    }
}

private static void insertDuplicate(Connection con) {
    System.out.println("\nTrying to insert product id 101 again:");
    String sql = "INSERT INTO products (id, name, category, price, stock) "
```

```
                + "VALUES (101, 'Duplicate', 'Test', 1.00, 1)";
            try (Statement st = con.createStatement()) {
                st.executeUpdate(sql);
            } catch (SQLException e) {
                printSqlError("Insert rejected", e);                    // the program keeps running
            }
        }

        private static void printSqlError(String title, SQLException e) {
            System.err.println(title + ":");
            System.err.println("  SQLState  : " + e.getSQLState());
            System.err.println("  ErrorCode : " + e.getErrorCode());
            System.err.println("  Message   : " + e.getMessage().split("; SQL statement")[0]);
        }
    }
```

Output

The program connected to the database, created the table, inserted five products, retrieved them and ran the filtered query. The last two lines show the duplicate insert being rejected and reported without stopping the program.

```
$ java -cp out:h2.jar com.simats.jdbc.ProductJdbcDemo
Connected to H2 2.2.220 (2023-07-04)
Table 'products' is ready.
5 products inserted and committed.

All products:
ID    Name                  Category             Price  Stock
101   Wireless Mouse        Electronics          799.00     50
102   Bluetooth Headphones  Electronics         2499.00     30
103   Cotton T-Shirt        Clothing             499.00    120
104   Steel Water Bottle    Home & Kitchen       349.00     75
105   Laptop Backpack       Accessories         1299.00     40

Electronics products costing less than Rs. 2000:
  Wireless Mouse         Rs. 799.00

Trying to insert product id 101 again:
Insert rejected:
  SQLState  : 23505
  ErrorCode : 23505
  Message   : Unique index or primary key violation: "PRIMARY KEY ON PUBLIC.PRODUCTS(ID) ( /*
key:101 */ 101, 'Wireless Mouse', 'Electronics', 799.00, 50)"
```

Figure 11: Output of ProductJdbcDemo

3.4 Exception Handling

Almost every JDBC call can fail because of a network problem, a wrong URL or password, invalid SQL or a violated constraint, so JDBC methods throw the checked exception `SQLException`. The program handles errors in the following ways.

- **Specific, informative catch blocks.** `ClassNotFoundException` is caught when the driver library is missing. `SQLException` provides the message, the standard five-character `SQLState` code and the vendor `ErrorCode`. SQLState classes are standardised: `08` means a connection failure, `23` an integrity-constraint violation and `42` a syntax error or access violation. Chained exceptions can be read with `getNextException()`.
- **Transactions.** Auto-commit is switched off for the batch insert. If any row fails, `rollback()` undoes the partial work so that the table never holds half of a batch. Auto-commit is restored in

a `finally` block.

- **Guaranteed cleanup.** Try-with-resources closes the `ResultSet`, the statement and the connection in every case, including when an exception occurs, so connections are never leaked.
- **Recovery and reporting.** The duplicate-key error (SQLState 23505 in H2) is caught locally, reported with its code and message, and the program continues. Exceptions are not swallowed silently. In a web application they would be logged, and the user would receive a friendly error page without the internal SQL details.